응대 리허설

AI 고객과 연습하는 기업용 상담 훈련 서비스

소속 및 반 광주 2반

고유번호 G053

이름 신서현

제출일 2026년 10월 23일

서비스 개요

신입 상담사가 까다로운 AI 고객과 상담을 연습하는 기업용 훈련 서비스다. 주문과 이전 상담 이력, 회사 정책을 확인해 해결안을 제시하면 실제 발언과 업무 자료에 근거한 평가 및 개선 답변을 받는다. 같은 상황에 재도전하고 직전 결과와 비교할 수 있다.

고객 역할과 평가·코칭을 분리하고, 정책 검색부터 원문 근거 확인까지 연결했다. 구현 범위는 단일 PC에서 실행하는 텍스트 상담 MVP이며 실제 주문 취소나 환급은 처리하지 않는다.

주요 구현 결과

영역	결과
상담 훈련	3개 시나리오, 스트리밍 대화, 동일 상황 재도전
근거 기반 평가	4개 항목 각 25점, 실제 발언·정책 연결, 개선 답변
정책 RAG	20개 정책 중 관련 4개 검색, OpenAI 임베딩과 Chroma
실행 검증	자동 테스트 33개 통과, 평가·코칭 성공 17.3초 기록
관리와 관측	관리자 실행 이력과 사용 토큰, LangSmith 기록 수집 확인

정책 판정 비교 실험에서 채팅 입력 토큰 합계는 18,909에서 6,388로 감소했다. 자체 작성 12개 사례의 결과이며 실제 교육 효과나 일반적인 정확도 향상을 의미하지 않는다.

검증 기준일 2026년 10월 22일 | 화면 시연 환경 Windows와 Python 3.14 | 평가 모델 gpt-5-mini

2 문제 개선 및 신규 구현 사항

해결하려는 문제

사람이 고객 역할을 맡는 상담 교육은 반복 연습에 인력이 필요하고, 같은 업무 조건을 유지하기 어렵다. 친절한 말투만 확인하면 잘못된 도착 약속이나 권한 밖 보상 안내를 놓칠 수도 있다. 이를 문제 가설로 두고 혼자 반복할 수 있는 상담과 근거를 확인할 수 있는 평가를 구현했다.

초기 구현에서는 모델이 근거 ID와 인용문을 직접 생성해 원문 검증이 실패할 수 있었다. 또한 윈도우 실행 중 정책 검색의 응답 대기가 100초를 넘어 평가 단계에 도달하지 못하는 문제가 발생했다.

문제	구현한 개선	확인 방법
반복 조건 변화	주문·이력·정책 스냅샷을 재도전에 복사	동일 스냅샷과 조회 기록 초기화 테스트
근거 없는 평가	모델은 E 참조 선택, 서버는 원문 연결	가짜 ID와 변조 인용문 거부 테스트
조회 여부 혼동	상담사와 평가자의 조회 기록 분리	평가자의 조회는 상담사 점수에 미반영
정책 증가	관련 정책 4개를 의미 검색해 주입	검색 ID·본문·거리와 버전 기록
검색 응답 대기	검색 프로세스 분리와 제한 시간 후 종료	지연시킨 실제 자식 프로세스 종료·복구 테스트
중복 입력	대화·근거 원문을 한 번씩 전달	모든 원문과 참조 보존 테스트

사용 흐름과 훈련 시나리오

시나리오 선택 후 주문과 이전 상담 이력을 조회하고 관련 정책을 검색한다. AI 고객과 상담을 마치면 네 항목의 점수와 이유, 근거 원문, 개선 답변을 확인한다. 같은 상황에 재도전하면 이전·현재 결과를 비교한다.

배송 지연과 도착 오안내, 제품 하자 반품, 이전 상담사의 권한 밖 보상 약속 등 세 가지 상황을 제공한다. 고객은 거친 말투와 압박을 유지하되 적절한 확인과 해결 절차에는 조건부로 수용하도록 설계했다.

3 적용 기술 요소

Multi Agent와 LangGraph

구성	역할	사용하는 정보
고객 Agent	고객 역할 유지와 스트리밍 답변	고객 설정과 실제 대화
평가 Agent	사실 확인·정책·해결안·의사소통 채점	대화, 조회 기록, 주문·이력, 검색 정책
코칭 Agent	강점과 다음 행동, 실제 발언의 개선 답변	같은 업무 근거와 평가 결과
서버 검증	인용과 개선 전 원문 검증, 총점 계산	확정 원문과 구조화된 모델 출력

LangChain의 역할별 Agent를 LangGraph로 연결했다. 상담 요청은 고객 Agent로, 평가 요청은 근거 수집·평가·코칭·검증으로 분기한다. 평가 Agent는 LLM-as-a-Judge 방식으로 채점한다. 별도 Supervisor LLM과 동적 재계획은 구현하지 않았으며, 작은 훈련 범위에서 호출 순서와 실패 지점을 예측할 수 있는 고정 워크플로를 선택했다.

MCP 표준 인터페이스

구분	인터페이스	실제 사용 범위
Tool	get_order · get_history	화면과 평가 그래프의 업무 조회
Tool	get_policy · search_policy	정책 목록 조회와 벡터 검색
Resource	policy://세션ID/정책ID	세션에 고정된 정책 원문 읽기
Prompt	policy_search_guide	외부 MCP 클라이언트용 검색 안내

FastMCP stdio 서버는 FastAPI lifespan에서 연결해 요청 간 재사용한다. 운영 그래프는 Tool을 명시적으로 호출한다. Resource와 Prompt는 SDK 통합 테스트로 제공과 조회를 확인했으며 운영 그래프의 자동 호출에는 포함하지 않았다.

화면과 API

Vue 화면은 상담, 업무 자료, 평가 결과와 관리자 현황을 제공한다. FastAPI는 REST API와 SSE 스트리밍을 제공하고 SQLite에 대화·조회·평가·실행 이력을 저장한다. 프런트엔드는 현재 단일 Vue SFC에 모여 있어 화면별 컴포넌트 분리는 후속 과제다.

서비스 아키텍처와 정책 검색

상담과 평가를 분기하고 평가 전에 정책 근거를 수집한다. 검색은 별도 프로세스에서 처리해 MCP의 통신 루프와 Chroma의 동기 작업을 분리했다.

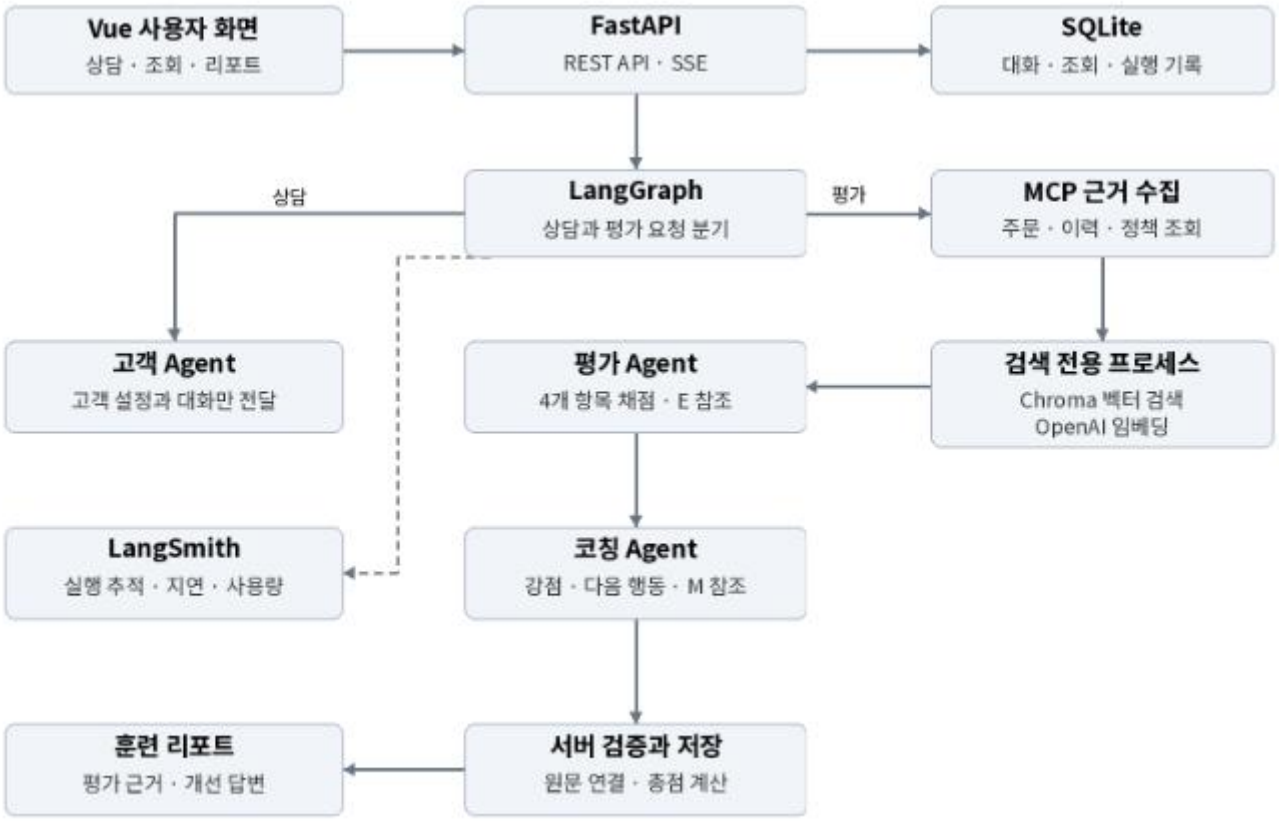


그림 1 구현된 서비스 흐름. 고객 응답은 SSE로 전달하며 점선은 LangSmith 실행 추적을 나타낸다.

정책 RAG와 원문 보존

짧은 정책 20개를 문서 단위 청크로 사용한다. OpenAI 임베딩으로 제목과 본문을 벡터화해 Chroma에 저장한다. 평가 시 최근 최대 6개 메시지에서 검색어를 구성하고 관련 정책 4개를 전달한다. 검색 결과는 생성문이 아닌 세션 스냅샷의 원문에 다시 연결한다.

문서 해시·임베딩 모드·모델이 달라지면 인덱스를 구분한다. 같은 검색어의 임베딩은 검색 프로세스가 유지되는 동안 재사용한다. 거리 값은 유사도 검색 결과이며 정책 준수 확률이나 채점 정확도가 아니다.

모듈화와 의존성 주입

API, DB, 스키마, Agent, MCP, RAG를 모듈로 나눴다. TrainingEngine에는 조회 도구를, PolicyRetriever에는 저장 경로와 임베딩 제공자를 주입해 실제 서비스 호출 없이도 검증할 수 있게 했다. Pydantic으로 입출력을 검사하고 세션 동시 요청과 최대 20턴을 제한한다.

4 구현 결과

업무 자료 조회와 정책 검색

배송 지연 시나리오에서 주문·상담 이력·정책을 조회한 뒤 출고 전 취소와 배송 지연 보상을 검색했다. 화면에는 전체 20개 정책 중 관련 4개를 찾은 결과와 정책 ID·원문·거리가 표시된다.



그림 2 실제 정책 검색 결과. DELAY-03은 배송비 3,000원 환급 접수와 추가 보상의 관리자 검토를 구분한다.

첫 상담 평가와 개선 답변

자료 조회 없이 사과와 대기 요청만 제시한 첫 상담은 38점을 받았다. 사실 확인은 조회 기반 상한에 따라 0점으로 제한됐으며, 나머지 항목은 발언과 정책에 대한 모델 판단을 반영했다.

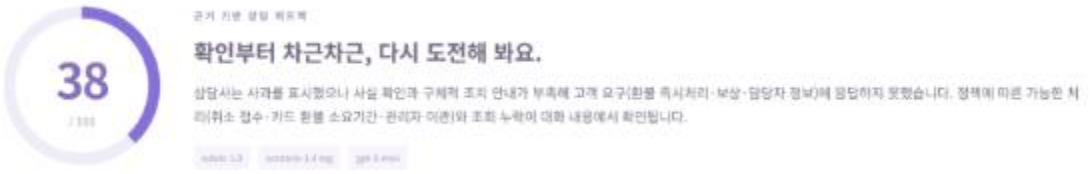


그림 3 첫 상담의 실제 평가 요약.

평가 항목	점수	확인된 행동과 누락
사실 확인	0 / 25	주문 상태와 이전 상담 이력 조회 누락
정책 준수	16 / 25	확정 보상은 피했으나 취소·환불·관리자 검토 안내 부족
해결안	10 / 25	실행 가능한 선택지와 다음 절차 부족
의사소통	12 / 25	사과는 했으나 고객 요구에 구체적으로 응답하지 못함

원래 답변은 “불편을 드려 죄송합니다. 배송이 지연되고 있으니 조금만 더 기다려 주세요”였다. 코칭은 주문·이력 확인과 취소·환불 절차, 권한 밖 보상의 관리자 검토를 구체화하도록 제안했다.

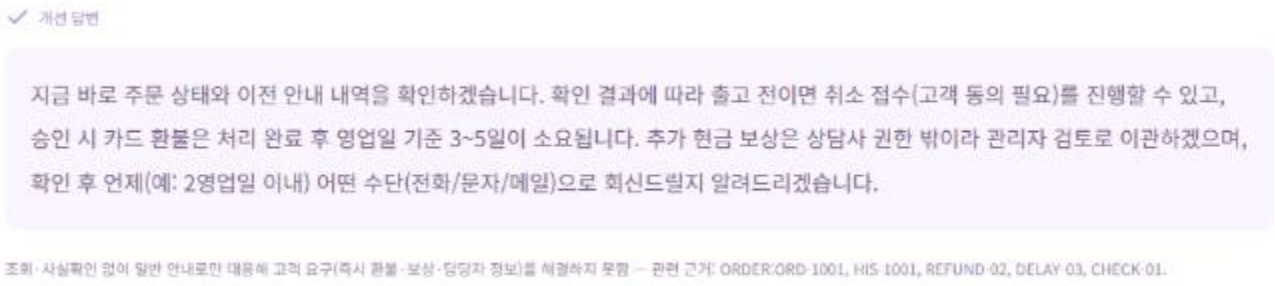


그림 4 실제 코칭 출력 일부. 회신 기한 예시의 정책 불일치는 교훈 절에서 분석한다.

평가 근거는 화면에서 펼쳐 확인할 수 있다. 서버는 모델이 선택한 E 참조를 원문에 연결하고, 코칭의 M 참조를 실제 상담사 발언에 연결한다. 이 검사는 원문의 존재와 일치를 확인하며 내용 해석의 정답까지 보장하지 않는다.

같은 상황 재도전과 개선 응대

재도전에서는 주문·이력·정책을 확인한 후 출고 전 상태, 3일 지연, 이전의 도착 확정 오안내를 설명했다. 개선 답변은 확인한 사실에 취소 접수 선택지와 승인 후 환불 소요 기간을 더 구체적으로 연결했다.

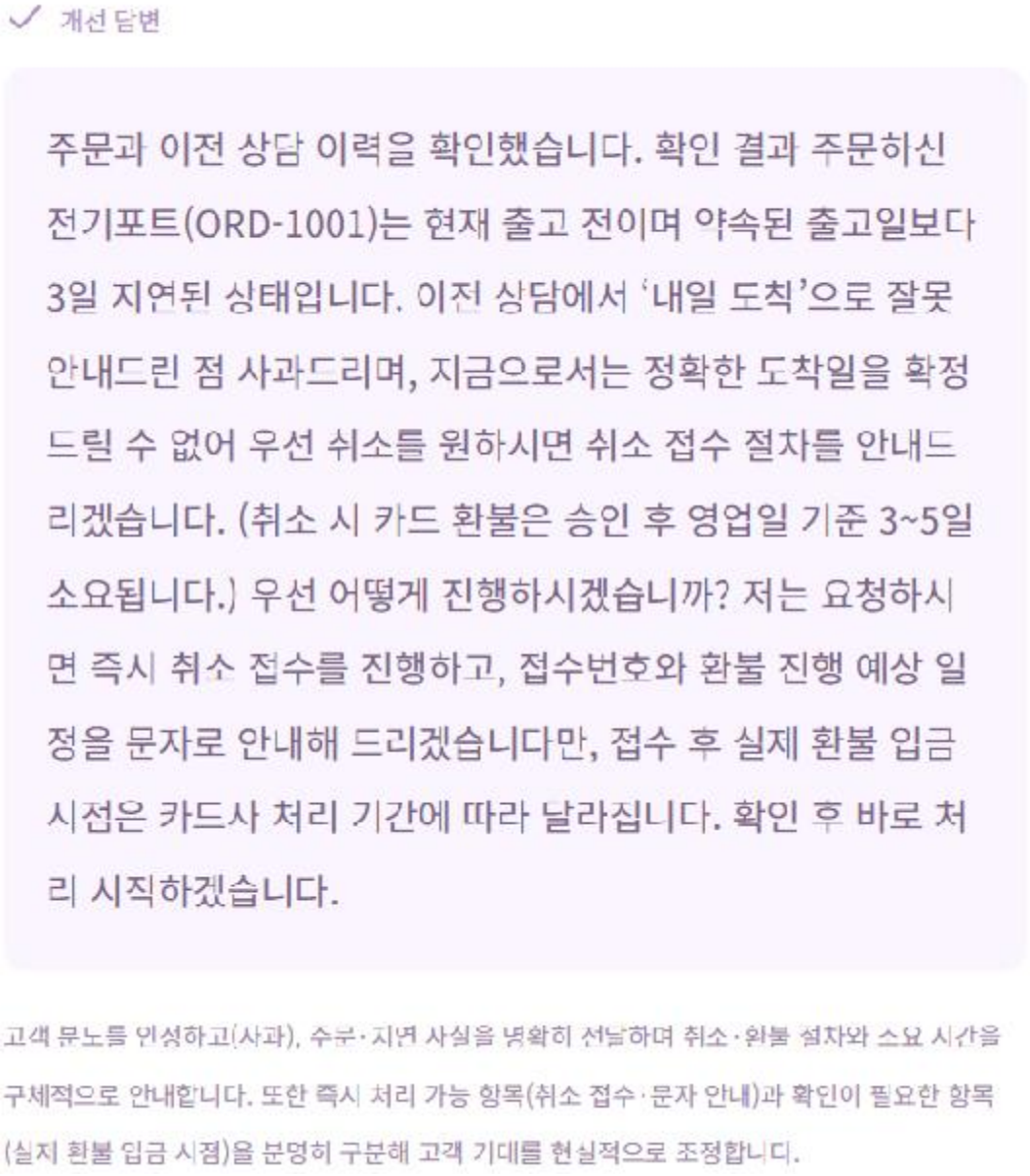


그림 5 재도전 상담의 실제 코칭 출력 일부. 최초 응대보다 사실 확인과 정책 절차가 구체적으로 표현돼 있다.

재도전은 직전 세션의 업무 스냅샷을 복사하고 조회 기록을 초기화한다. 모델·평가 기준·프롬프트 버전이 같은 완료 세션끼리 이전·현재 점수를 비교한다. 이 비교 기능은 자동 테스트로 확인했다. 시연용으로 응대 내용을 바꾼 사례이므로 실제 교육 효과의 인과적 증거로 해석하지 않는다.

관리자 현황과 실행 기록

관리자 화면은 훈련 건수와 완료 건수, 평균 점수와 오류를 집계하고 개별 실행의 상태·소요 시간·사용 토큰을 제공한다. 실행 ID와 상담 ID로 상세 기록을 연결한다.



그림 6 2026년 10월 22일 관리자 현황. 누적 오류에는 이전 실패 기록이 포함된다.

항목	화면 확인값	해석
전체 훈련	6건	현재 로컬 DB에 저장된 훈련
완료한 훈련	4건	평가 결과가 저장된 훈련
평균 점수	43.8점	완료된 여러 훈련의 평균
최근 실행 오류	2건	누적 실행 오류이며 현재 실패 여부와 구분
최근 평가·코칭	success · 17.3초	평가·코칭 요청 전체의 완료 기록
채팅 사용 토큰	6,442	해당 평가·코칭의 SDK 집계 총 토큰

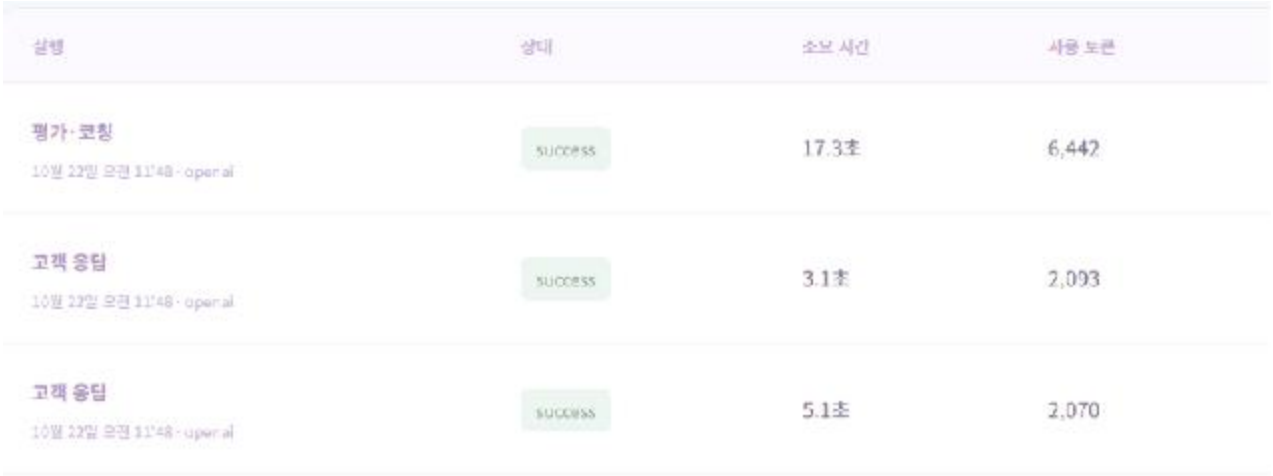


그림 7 최근 실행 기록의 핵심 열. 임베딩 사용량은 리포트의 검색 메타데이터에 별도로 저장한다.

이전에는 정책 검색 응답 대기 중 100.1초에 실패했다. 이후 시연에서 평가·코칭의 정상 완료를 확인했다. 실패 기록과 성공 기록의 입력·캐시 상태가 통제되지 않았으므로 이를 속도 개선율로 계산하지 않는다.

LangSmith 관측 결과

LangSmith 추적 설정을 활성화한 뒤 프로젝트에 최근 실행이 수집되는 것을 확인했다. 화면의 집계는 최근 7일 동안 프로젝트에 기록된 추적 결과이며 단일 상담의 성능 지표가 아니다.



그림 8 LangSmith 추적 집계. 프로젝트에 성공 40건과 실패 1건이 표시된다.



그림 9 프로젝트 추적 지연 집계 P50 5.2초와 P99 33초. 평가·코칭만의 지연으로 해석하지 않는다.

관리자 화면의 오류 2건과 LangSmith의 실패 1건은 수집 시작 시점과 집계 범위가 달라 직접 비교할 수 없다. 추적 설정 활성화 여부와 실제 기록 수집 여부를 함께 확인해야 한다는 점을 확인했다.

정책 판정 비교 실험

동일한 20개 정책을 모두 전달한 Before와 관련 상위 4개를 검색한 After를 비교했다. Before는 통제 실험용 기준선이다. 초기 앱의 시나리오별 정책 3~4개 직접 연결 방식에 대한 실측값으로 설명하지 않는다.

자체 작성 12개 사례를 조건별 한 번씩 실행하고 순서를 교차 배치했다. 판정 모델은 gpt-5-mini, reasoning_effort는 low, 임베딩은 text-embedding-3-small이다. 결과 저장 시각은 2026년 10월 22일 10시 09분 KST이며 Python 3.14.0에서 실행했다.

지표	Before 전체 문서	After 검색 문서
정상 완료	12 / 12	12 / 12
평균 주입 정책	20개	4개
기대 정책 포함률	전체 범위 100%	Recall@4 100%
기대 정답 일치율	91.7% 11 / 12	100% 12 / 12
평균 처리 시간	4.144초	3.774초
채팅 입력 토큰 합계	18,909	6,388
채팅 출력 토큰 합계	3,949	3,322
채팅 총 토큰 합계	22,858	9,710
임베딩 토큰 합계	0	1,882

입력 토큰 합계는 66.2%, 채팅 총 토큰은 57.5%, 평균 처리 시간은 8.9% 감소했다. 시간은 문맥 준비·검색·정책 판정의 합계이며 전체 상담 평가·코칭 시간과는 다르다. 임베딩은 최초 문서 1,524토큰과 질의 358토큰을 별도로 집계했다.

Before의 전체 범위 포함률은 모든 정책을 준 결과다. After의 Recall@4는 기대 정책이 상위 4개에 포함됐는지를 뜻한다. 기대 정답 일치율은 자체 라벨과의 일치율이며 전문가가 검증한 정확도가 아니다. 채팅과 임베딩의 단가가 다르므로 토큰 감소율을 비용 절감률로 바꾸지 않았다.

B04에서는 두 조건 모두 CHECK-01을 포함했지만 조회 언급 누락에 대한 모델 해석이 달라 판정이 바뀌었다. 따라서 한 건의 차이를 검색 효과로 단정할 수 없다. 이 실험은 검색 프로세스 분리 패치 이전 코드로 수행했으며 현재 버전의 지연 개선 수치로 사용하지 않는다.

자동 검증과 오류 대응

검증 항목	결과	검증 범위
자동 테스트	33 passed	기존 28개와 추가 회귀 테스트 5개
MCP 통합	통과	실제 stdio Tool · Resource · Prompt와 업무 조회
검색 연결	통과	실제 Chroma와 로컬 모의 OpenAI 임베딩 서버
검색 중단 복구	통과	지연시킨 실제 OS 자식 프로세스 종료와 다음 검색 복구
입력 중복 제거	통과	전체 원문과 평가 근거 참조 보존
출처 검증	통과	가짜 ID · 변조 인용 · 개선 전 발언 위조 거부
사용자 시연	성공 기록	Windows의 정책 검색 · 평가 · 코칭 · 관리자 화면`

자동 테스트는 Linux와 Python 3.12에서 실제 MCP · SQLite · Chroma를 사용해 수행했다. 공급자 호출은 모의 응답으로 대체해 구조와 오류 처리를 검증했다. 사용자 시연은 Windows와 실제 OpenAI API로 수행했으며 두 검증 범위는 구분한다.

정책 검색 시간 초과 대응

Traceback에서 mcp_evidence가 search_policy 응답을 기다리다 100초 제한에 걸린 것을 확인했다. 기존 기록으로는 Chroma 내부 작업과 임베딩 요청 중 어느 부분이 원인인지 확정할 수 없었다. 이를 구분하기 위해 단계별 상태를 남기고 검색 전용 프로세스를 재사용하도록 수정했다.

검색이 45초를 초과하면 해당 프로세스를 종료하고 다음 검색에서 다시 생성한다. 임베딩 요청은 15초, SDK 자동 재시도는 0회로 설정했다. 이전 인덱스는 보존하고 새 실행 캐시를 사용한다. 검색 실패 시 점수를 임의 생성하지 않고 대화를 유지한다.

평가와 코칭의 호출량 조정

대화와 근거가 여러 필드에 중복되던 입력을 정리해 원문을 한 번만 전달한다. 핵심 개선 답변 한 개를 요청하고 출력 상한을 평가 6,500에서 3,800, 코칭 5,000에서 2,800으로 줄였다. 각 호출은 45초로 제한하되 근거 · 출력 형식 오류에 대한 한 번의 교정 기회와 최종 원문 검증은 유지했다.

출력 상한 감소는 실제 토큰 감소율과 다르며 긴 응답이 잘리거나 모델 품질이 달라질 수 있다. 실제 절감률과 결과 품질은 새 버전의 사용량 및 반복 실행 결과로 확인해야 한다.

5 교훈

한계와 트레이드오프

선택	얻은 점	남은 한계
고정 LangGraph	호출 순서와 실패 지점 파악	Supervisor와 동적 재계획 미지원
3개 역할 Agent	고객·평가·코칭의 책임 분리	별도 모델 호출의 비용과 지연
서버 원문 연결	인용문 생성 오류 방지	원문의 잘못된 해석까지 검증하지 못함
상위 4개 검색	입력 문맥과 정책 수 제한	필요 정책의 검색 누락 가능
조회 기반 상한	관찰 가능한 조회 행동 반영	자료를 읽고 이해했는지는 별도 평가 필요
출력량 제한	짧은 코칭과 호출량 통제	응답 잘림과 설명 부족 가능

실행에서 확인한 코칭의 한계

첫 상담 코칭은 관리자 회신 기한에 2영업일 이내라는 예시를 제시했다. 훈련 정책 PROMISE-05의 회신 목표는 영업일 1일 이내이므로 일치하지 않는다. 또한 정책에 없는 연락 수단을 확정하는 표현은 별도 검토가 필요하다. 이는 실제 원문을 인용할 수 있어도 개선 답변의 모든 내용이 정책에 맞는다고 보장할 수 없다는 사례다.

향후 개선 사항

우선 금액·기한·권한을 구조화된 정책 값으로 검증하고 코칭에서 확정 표현을 검사해야 한다. 이후 전문가의 독립 채점, 더 많은 사례와 반복 실행으로 모델 변동을 측정할 계획이다. 여러 정책이 함께 필요한 질문과 검색 실패 사례를 추가하고 하이브리드 검색·재순위화도 비교할 수 있다.

현재는 단일 PC의 로컬 MVP다. 사용자 로그인, 기업별 문서·상담 접근 권한, 운영 배포와 HTTPS, 백업 및 보존 정책은 후속 범위다. MCP와 검색용 자식 프로세스를 사용하지만 분산 운영을 검증한 서비스는 아니다.

수행 소감

구현 과정에서 모델에 판단을 맡길 부분과 서버가 보장할 부분을 나누는 일이 중요했다. 원문 연결과 총점 계산은 서버가 책임지도록 바꾸면서 오류를 구분하기 쉬워졌다. 실제 시연에서는 근거 연결이 정상이어도 코칭의 기한이 틀릴 수 있어 의미 검증이 추가로 필요하다는 점을 확인했다.

또한 추적 설정을 켜는 것과 실행 기록이 수집되는 것은 별개였다. 성공 화면뿐 아니라 실패 단계와 사용량을 함께 남겨야 지연과 품질 문제를 구체적으로 설명할 수 있었다.

구현 근거 backend/app/agents.py · mcp_client.py · mcp_server.py · rag.py · rag_worker.py · main.py
검증 근거 backend/tests · scripts/benchmark_rag.py · rag-benchmark.json · 2026년 10월 22일 실행 화면